

Track A: Building the Chat Screen (Frontend)

Your job this week, in one sentence

Build the chat screen people will actually see and use — but instead of connecting it to the real AI model right away, you'll fake it, so you can move fast without waiting on anyone else.

Why fake it first?

Think of it like building a car's dashboard before the engine is finished. You can still test that the buttons, lights, and screens all work — you just feed them a fake signal for now. Later, swapping the fake signal for the real engine is easy, because everything around it already works.

A few words you'll see, explained simply

- **Mock / fake data** — pretend responses you write yourself, used to test the app before the real backend exists.
- **Streaming** — when text appears bit by bit, like someone typing live, instead of popping up all at once. That's what makes a chatbot feel alive instead of just dumping a wall of text.
- **Component** — a reusable piece of a webpage, like a Lego brick. The message bubble, the input box, the send button are all separate components.

Your steps, in order

1. Set up your project. Run:

```
npm create vite@latest
```

Choose React. Add Tailwind for styling so you're not writing CSS from scratch. This gives you an empty app to build on.

2. Build the pieces of the chat screen. At minimum, you need: - A box showing the conversation (MessageList) - A single message bubble (MessageBubble) - A text input + send button (ChatInput) - A button to attach a file (FileAttachButton) — for now it just needs to look right and remember which file was picked. It doesn't need to upload anything yet.

3. Fake the AI's reply — no backend needed. Write a small function that takes a hardcoded sentence and reveals it word by word, with a tiny pause between each word. This stand-in "pretend brain" is what you'll build and test against until Track B is ready.

4. Connect the pieces. When someone hits send: - Show their message immediately. - Call your fake-reply function. - As each word comes back, add it to the assistant's message bubble. - When it's finished, let the person type again.

5. Make it feel polished. - Show a subtle "thinking" indicator while the fake reply streams in. - Disable the input while it's "thinking." - Add a friendly empty state, like "Ask me anything!", when there's no

conversation yet. - Check it looks fine on a phone-sized screen, not just your laptop.

6. Leave yourself a clear note. Somewhere obvious in the code, write: `// TODO: replace this fake function with a real fetch() call to the backend` That's the exact spot you'll edit once Track B's backend is live.

How you'll know you're done

- ☐ Typing a message shows it instantly.
- ☐ A fake reply "types itself" out, word by word.
- ☐ The input disables while the fake reply streams, and re-enables after.
- ☐ Picking a file shows it as "attached," even though nothing happens with it yet.
- ☐ The app looks decent on a phone-sized screen.

What you don't need to worry about yet

- Real file uploads
- Real answers from a real model
- Deployment
- CORS errors — that's Track B's headache this week, not yours